

Think of MVVM + Clean Architecture like **a restaurant**.

1. The Simple Idea

Your Android app has **3 main rooms**.

APP



That's it.

2. What Each Room Does

A. Presentation → The UI Room

This is **what the user sees**.

Contains:

- Screens
- ViewModels
- UI State

Example:

```
presentation/
  HomeScreen.kt
  HomeViewModel.kt
```

Example

HomeScreen.kt

```
@Composable
fun HomeScreen() {

    Text("Hello Student")
}
```

```
}
```

This displays UI.

HomeViewModel.kt

```
class HomeViewModel : ViewModel() {  
  
    val title = "Hello Student"  
  
}
```

This prepares data for the screen.

Think:

Presentation = Screen + ViewModel

B. Domain → The Brain Room

This contains **app rules**.

Example:

"Get all students."

"Login user."

"Calculate total price."

Contains:

- models
- usecases
- repository interfaces

Example:

```
domain/  
    Student.kt  
    GetStudentsUseCase.kt
```

Student Model

```
data class Student(  
    val name: String  
)
```

Use Case

```
class GetStudentsUseCase {  
  
    fun execute() {  
  
    }  
  
}
```

Think of UseCase as:

ONE APP ACTION

Examples:

LoginUser
GetStudents
DeleteStudent
AddProduct

C. Data → The Storage Room

This handles **where data comes from**.

Examples:

- API
- Room Database
- Firebase

Contains:

```
data/  
    repository/  
    remote/  
    local/
```

Example Repository:

```
class StudentRepository {
```

```
fun getStudents(): List<Student> {  
  
    return listOf(  
        Student("John"),  
        Student("Mary")  
    )  
  
}  
  
}
```

Think:

Data layer = Fetch data

3. Simple Flow

Suppose user opens a student screen.

Flow becomes:

Screen
↓
ViewModel
↓
UseCase
↓
Repository
↓
API / Database

Think like ordering food:

Customer
↓
Waiter
↓
Chef
↓
Store

Android equivalent:

UI
↓
ViewModel

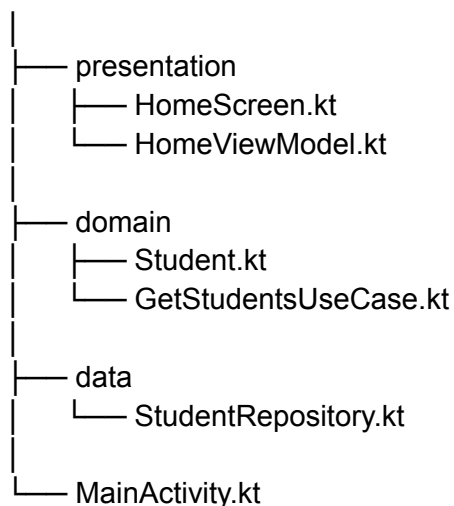
↓
UseCase
↓
Repository
↓
API

4. SIMPLE Beginner Folder Structure

Forget the huge structure first.

Start with THIS:

com.example.app



Only **3 folders**.

Easy.

5. Real Example

Suppose you build a **Student App**.

Step 1 — Model

domain/Student.kt

data class Student(

```
    val name: String  
  )  
}
```

Step 2 — Repository

data/StudentRepository.kt

```
class StudentRepository {  
  
    fun getStudents(): List<Student> {  
  
        return listOf(  
            Student("John"),  
            Student("Mary")  
        )  
    }  
}
```

Step 3 — UseCase

domain/GetStudentsUseCase.kt

```
class GetStudentsUseCase(  
    private val repository: StudentRepository  
) {  
  
    fun execute() =  
        repository.getStudents()  
  
}
```

Step 4 — ViewModel

presentation/HomeViewModel.kt

```
class HomeViewModel : ViewModel() {  
  
    private val repository =  
        StudentRepository()  
  
    val students =  
        repository.getStudents()  
  
}
```

```
}
```

Step 5 — Screen

presentation/HomeScreen.kt

```
@Composable
fun HomeScreen(
    viewModel: HomeViewModel
){

    LazyColumn {

        items(viewModel.students) {

            Text(it.name)

        }

    }

}
```

6. The Memory Trick

Remember this:

Folder	Job
presentation	Show UI
domain	App logic
data	Fetch data

Short version:

presentation = display
domain = think
data = fetch

7. Ignore These For Now

Do NOT stress about:

- ✗ DTOs
- ✗ Mappers
- ✗ RepositoryImpl
- ✗ NetworkModule
- ✗ Advanced DI

Master first:

- ✓ Composable
- ✓ ViewModel
- ✓ Repository
- ✓ State

After that, Clean Architecture becomes much easier.

Your beginner roadmap should be:

1. Compose UI
2. State
3. ViewModel
4. Repository
5. MVVM
6. Clean Architecture
7. DI (Koin/Hilt)